Validación Cruzada

Una simulación interminable donde cada dataset es otra píldora roja y los errores surgen como glitches por copiar código de ChatGPT

Daniel Barajas Higuera

MLEARN

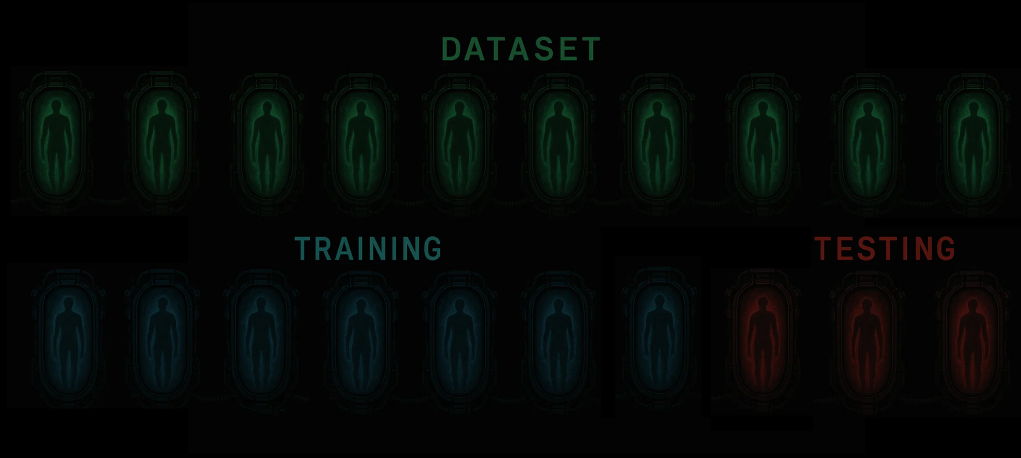


Contenido

1. Introducción a la validación cruzada
2. Validación cruzada
3. Data Leakage

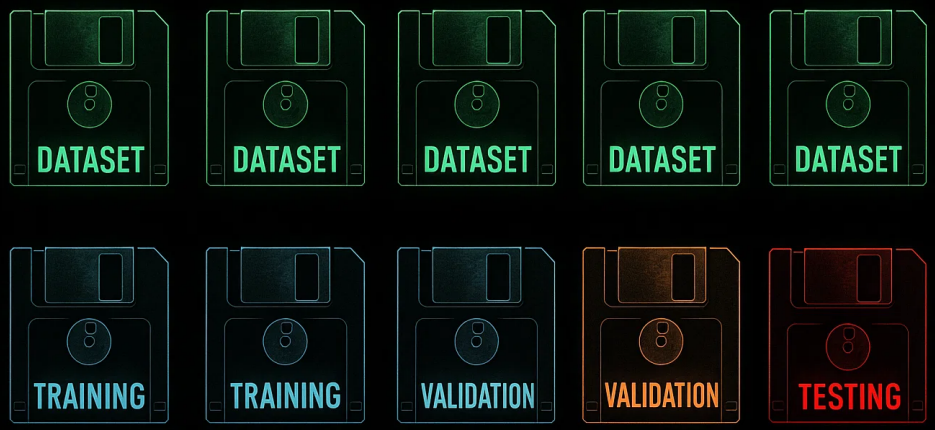
¿Por qué separar datos?

- Entrenar y probar con los mismos datos es autoengaño: el modelo puede memorizar y fingir que sabe.
- Train/Test: 70 % para aprender, 30 % para medir desempeño en datos nuevos.
- Para modelos más complejos, añadimos un conjunto de validación para ajustar parámetros.



Entrenamiento, validación y prueba

- Entrenamiento: el modelo aprende.
- Validación: se ajustan hiperparámetros sin tocar la prueba.
- Prueba: mide el resultado final, sin más ajustes.
- Ejemplo clásico: 60 % entrenamiento, 20 % validación, 20 % prueba.



Comparación de enfoques

Método	Entrenamiento	Validación	Prueba
Train-Test	70 %	–	30 %
Train-Val-Test	60 %	20 %	20 %

Contenido

1 Introducción a la validación cruzada

2 Validación cruzada

3 Data Leakage

Validación cruzada

En lugar de separar una sola vez, se realizan varias particiones de los datos. Cada partición toma turno como conjunto de prueba mientras el resto se usa para entrenar. Se promedian los resultados para tener una mejor estimación.

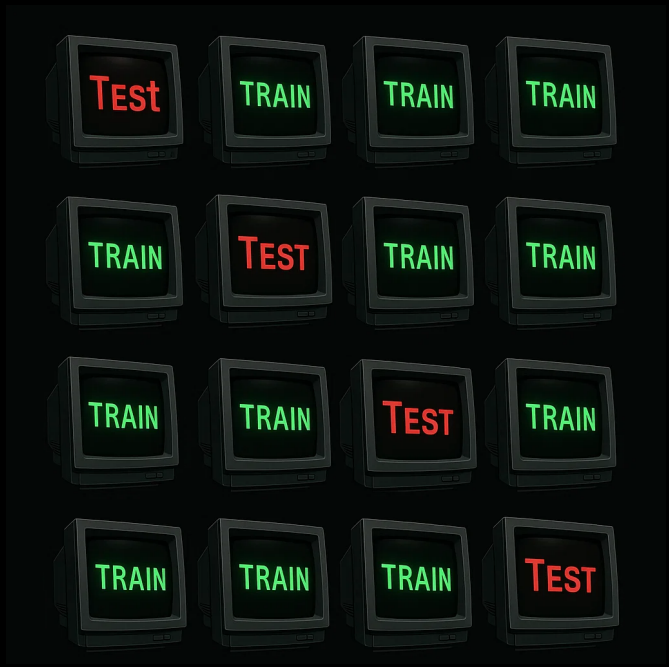
- Todos los datos se usan para entrenar y probar.
- Se repite el proceso múltiples veces.
- Ideal para conjuntos pequeños o cuando se requiere estabilidad.



¿Por qué usar validación cruzada?

Con una sola división podríamos tener suerte (o mala suerte) con los datos de prueba. La validación cruzada reduce ese riesgo, promediando el rendimiento del modelo en distintos subconjuntos. Es una forma más justa de evaluar.

- Reduce el impacto de una partición sesgada.
- Usa mejor toda la información disponible.
- Ayuda a elegir el mejor modelo.



Validación cruzada: analogía en Matrix

Neo no se enfrenta a un solo Agente Smith. En *Matrix Reloaded*, debe luchar contra muchos clones en distintas situaciones.

Una sola prueba podría no ser suficiente. Si Neo venciera a un solo Smith, podríamos sobreestimar su habilidad.

- Cada clon de Smith es una partición distinta.
- Neo (el modelo) se entrena con los demás datos.
- Se evalúa en cada clon por separado.
- Se promedia el desempeño.



¿Por qué varios Smiths?

La validación cruzada asegura que su rendimiento sea consistente, sin depender de una sola situación favorable o desfavorable.

- Evalúa la estabilidad del modelo.
- Reduce el riesgo de una división con suerte.
- Usa toda la información disponible.
- Mejora la confianza en el modelo final.



Validación cruzada con diamonds

```
1  import seaborn as sns, pandas as pd
2  from sklearn.linear_model import LinearRegression
3  from sklearn.model_selection import cross_val_score
4  from sklearn.metrics import make_scorer, mean_squared_error
5
6  df = sns.load_dataset('diamonds')
7  df['volume'] = df['x'] * df['y'] * df['z']
8
9  X = df[['carat', 'depth', 'table', 'volume']]
10 y = df['price']
11 modelo = LinearRegression()
12
13 # Validación cruzada con 5 particiones (folds)
14 scores = cross_val_score(modelo, X, y,
15                 scoring=make_scorer(mean_squared_error,
                   ↪  squared=False),
16                 cv=5)
17 print(f"RMSE promedio: {scores.mean():.2f}")
```

¿Qué hace cross_val_score?

- Divide los datos en `cv` partes (aquí, 5).
- Entrena y evalúa el modelo 5 veces, usando cada parte como prueba una vez.
- `make_scorer` permite usar RMSE como métrica personalizada.
- Devuelve un arreglo con el error de cada ciclo.
- Se promedia para estimar el desempeño general.

```
1 scores = cross_val_score(modelo, X, y,
2                          scoring=make_scorer(mean_squared_error,
                           ↪  squared=False),
3                          cv=5)
```

Comparación de enfoques

```
11110000 01010101 10101010 00110011 11001100 00000000 11111111 00001111 11110000
11000011 00111100 01011010 10100101 01101001 10010110 00100111 11011011 10101010
10100101 01011010 11111111 00000000 10011001 01100110 11110000 00001111 11000011
11110000 01010101 10101010 00110011 11001100 00000000 11111111 00001111 11110000
11000011 00111100 01011010 10100101 01101001 10010110 00100111 11011011 10101010
```

Método	Entrenamiento	Validación	Prueba
Train-Test	70 %	–	30 %
Train-Val-Test	60 %	20 %	20 %
K-Fold (5)	80 % aprox.	Rotativa	20 % aprox.


```
11110000 01010101 10101010 00110011 11001100 00000000 11111111 00001111 11110000
11000011 00111100 01011010 10100101 01101001 10010110 00100111 11011011 10101010
10100101 01011010 11111111 00000000 10011001 01100110 11110000 00001111 11000011
11110000 01010101 10101010 00110011 11001100 00000000 11111111 00001111 11110000
11000011 00111100 01011010 10100101 01101001 10010110 00100111 11011011 10101010
```


Cada enfoque tiene un costo y un beneficio: robustez vs simplicidad.

Contenido

1 Introducción a la validación cruzada

2 Validación cruzada

3 Data Leakage

¿Qué es *data leakage*?

- Es cuando información del conjunto de **validación/prueba** se filtra hacia el **entrenamiento**.
- Consecuencia: métricas infladas en validación/prueba y caída drástica en datos realmente nuevos.
- Regla de oro: **primero dividir**, **luego** calcular transformaciones con el **entrenamiento** y aplicarlas al resto.



Fuentes comunes de leakage

- **Escalado/normalización global**: usar media y desviación de *toda* la base.
- **Imputación global**: llenar nulos con estadísticas calculadas con todo el dataset.
- **Selección de variables** (correlaciones, tests) hecha antes de separar los datos.
- **Ingeniería de variables con info futura**: usar campos que no existirían al predecir.
- **Temporal**: mezclar pasado y futuro en problemas con fecha/hora.





Nota diamonds: crear `volume = x*y*z` no es leakage; usar `price` para crear características sí lo sería.

Ejemplo incorrecto (sí, esto fuga)

```
1  import seaborn as sns
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import StandardScaler
4
5  df = sns.load_dataset('diamonds')
6  df['volume'] = df['x'] * df['y'] * df['z']
7  X = df[['carat','depth','table','volume']]
8  y = df['price']
9
10 # Escalado antes de separar:
11 #   calcula estadísticas con TODO el dataset
12 scaler = StandardScaler()
13 X_scaled = scaler.fit_transform(X)  # <-- fuga
14
15 X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
   ↪  test_size=0.3, random_state=42)
16
17 # Entrenar aquí... las métricas estarán optimistas por fuga.
18
```

Ejemplo correcto con Pipeline

```
1
2   df = sns.load_dataset('diamonds')
3   df['volume'] = df['x'] * df['y'] * df['z']
4   X = df[['carat','depth','table','volume']]
5   y = df['price']
6
7   # Primero se divide el dataset:
8
9   X_train, X_test, y_train, y_test = train_test_split(
10      X, y, test_size=0.3, random_state=42)
11
12  # Continúa ...
13
```

Ejemplo correcto con Pipeline

```
1
2   ### ...
3   # Primero se divide el dataset:
4   X_train, X_test, y_train, y_test = train_test_split(
5       X, y, test_size=0.3, random_state=42)
6
7   # Se escala y se modela
8   pipe = Pipeline([
9       ('scaler', StandardScaler()),
10      ('model', LinearRegression())
11  ])
12
13  pipe.fit(X_train, y_train)
14  # calcula estadísticas SOLO con train
15
16  y_pred = pipe.predict(X_test)
17  rmse = mean_squared_error(y_test, y_pred, squared=False)
18  print(f"RMSE prueba: {rmse:.2f}")
19
```

Leakage menos obvio

- **Selección de características**: elegir columnas por correlación usando todo el dataset. Solución: selección dentro de `Pipeline` o con CV.
- **Codificación de categorías**: ajustar `OneHotEncoder` con todo el dataset. Solución: encodificar tras el split o en `Pipeline`.
- **Temporal**: barajar sin respetar tiempo. Solución: `TimeSeriesSplit` o cortes por fecha.
- **Target leakage**: variables que llevan la respuesta implícita (p. ej., "precio con descuento" para predecir "precio"). Elimine o redefina.

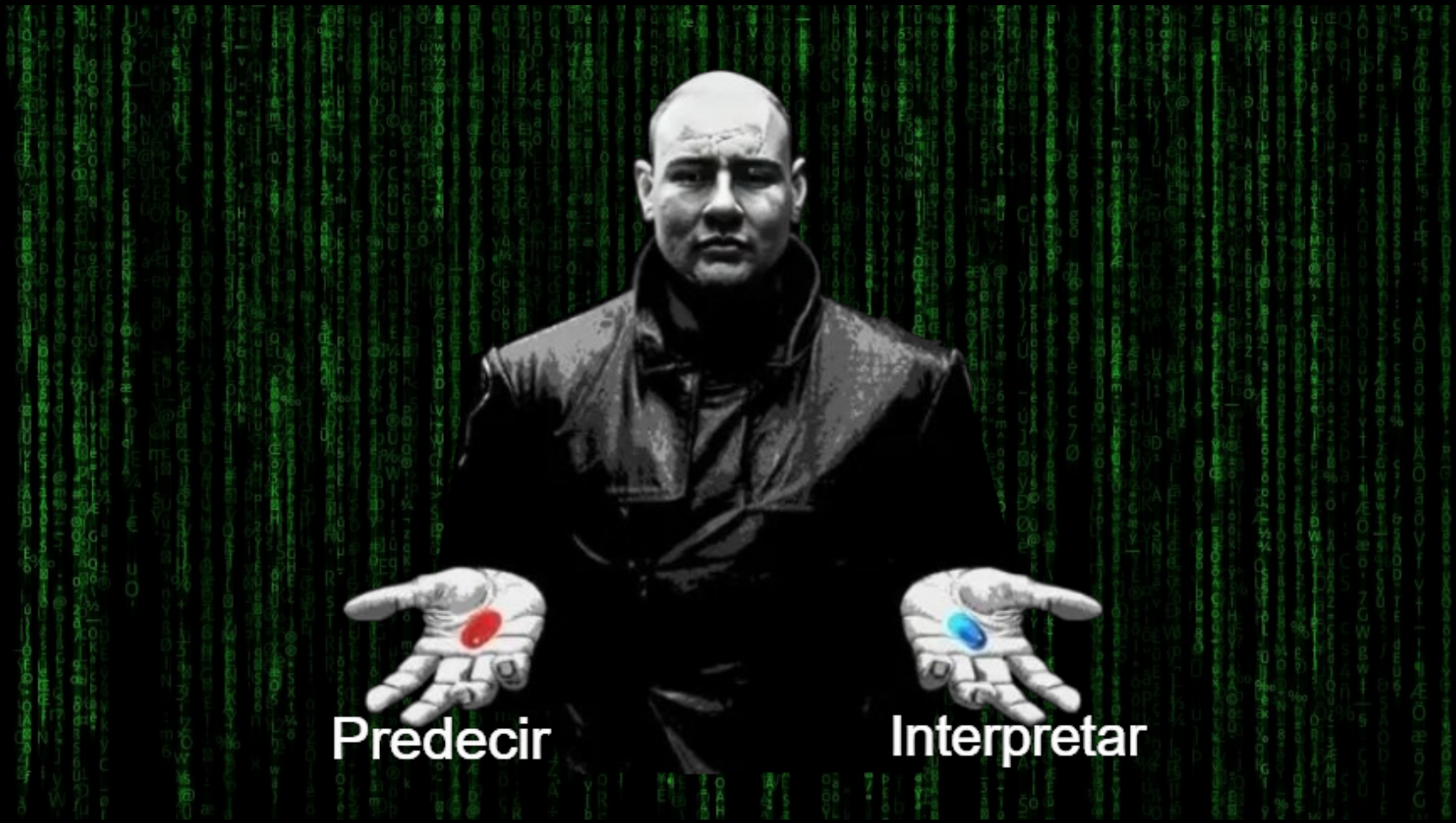
Checklist rápido en la Matrix

- ¿Dividiste primero, antes de transformar? Si no lo hiciste, aún vives en la ilusión.
- ¿Guardaste todas tus habilidades dentro del **dojo** (Pipeline), o dejaste tu entrenamiento disperso como fragmentos de código roto?
- ¿Tus batallas con los clones de Smith se hacen en la **validación cruzada**, o buscaste atajos en el Oráculo para engañarte a ti mismo?
- ¿Respetaste el flujo del tiempo, o lo alteraste mezclando pasado y futuro como si la Matrix se hubiera corrompido? (`TimeSeriesSplit`).
- ¿Permitiste que el enemigo se infiltrara en tu entrenamiento con pistas ocultas del objetivo? Ese es el glitch más peligroso.

Si dudas de alguna respuesta... la Matrix ya te controla.

El dilema con los datos

10101010 00011101 11001100 01010101 11110000 00110011 10011001 01100110 00001111 11100000



11001100 01010101 11110000 00110011 10011001 01100110 00001111 11100000 10100101 01011010